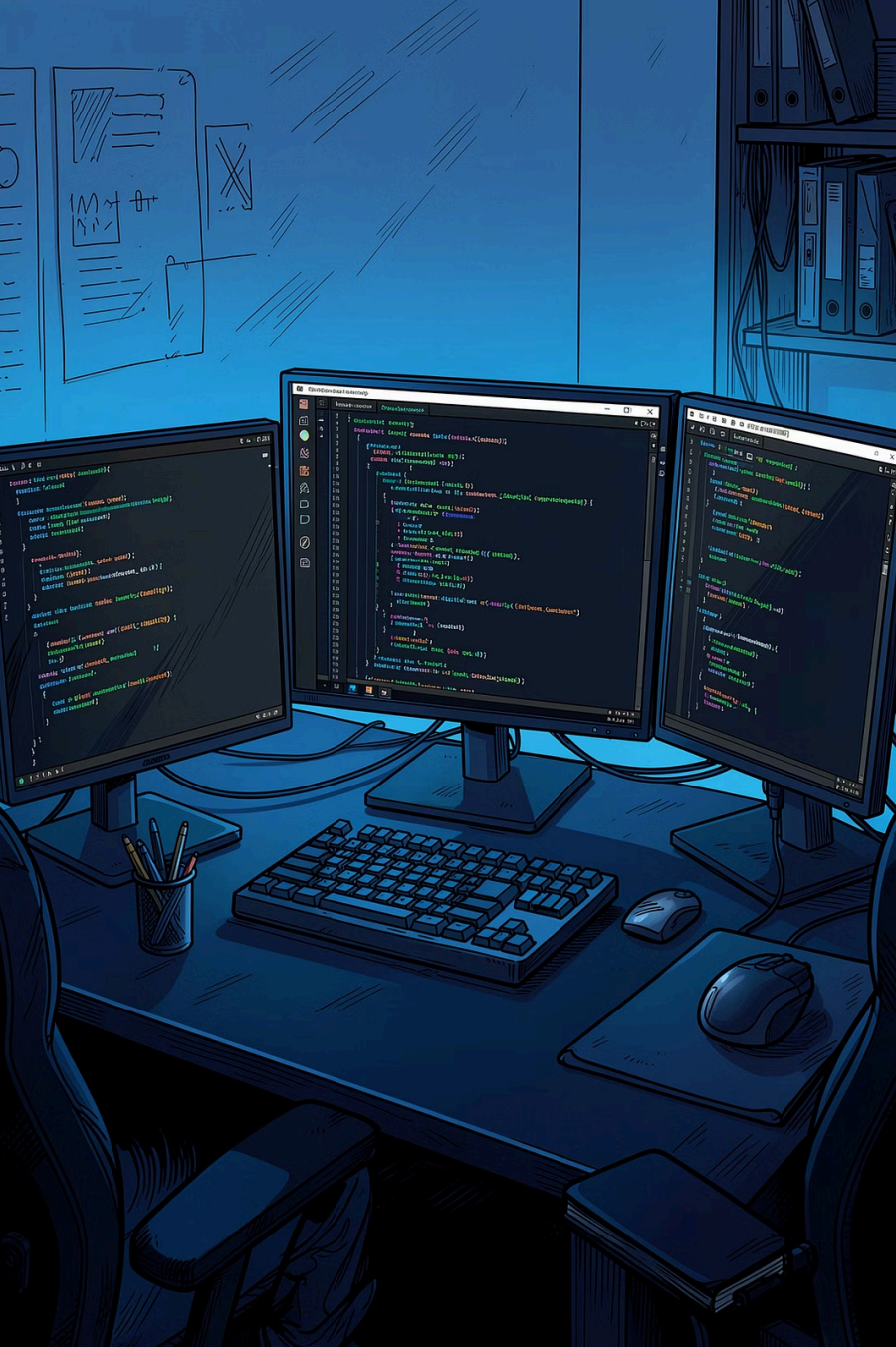




BACKEND DEVELOPMENT · INFOSEC

Testing Fundamentals

Writing code is easy. **Maintaining it is hard.** Without testing, bugs reach production, refactoring becomes dangerous, and new features silently break old ones. Testing builds confidence — in your code, your team, and your deployments.

What Is Testing?

Testing verifies that software behaves as expected — before it reaches production. It answers three critical questions every engineer should ask.



Does it work?

Verify correct behavior under normal, expected conditions.



Does it handle bad input?

Validate edge cases, malformed data, and unexpected user behavior.



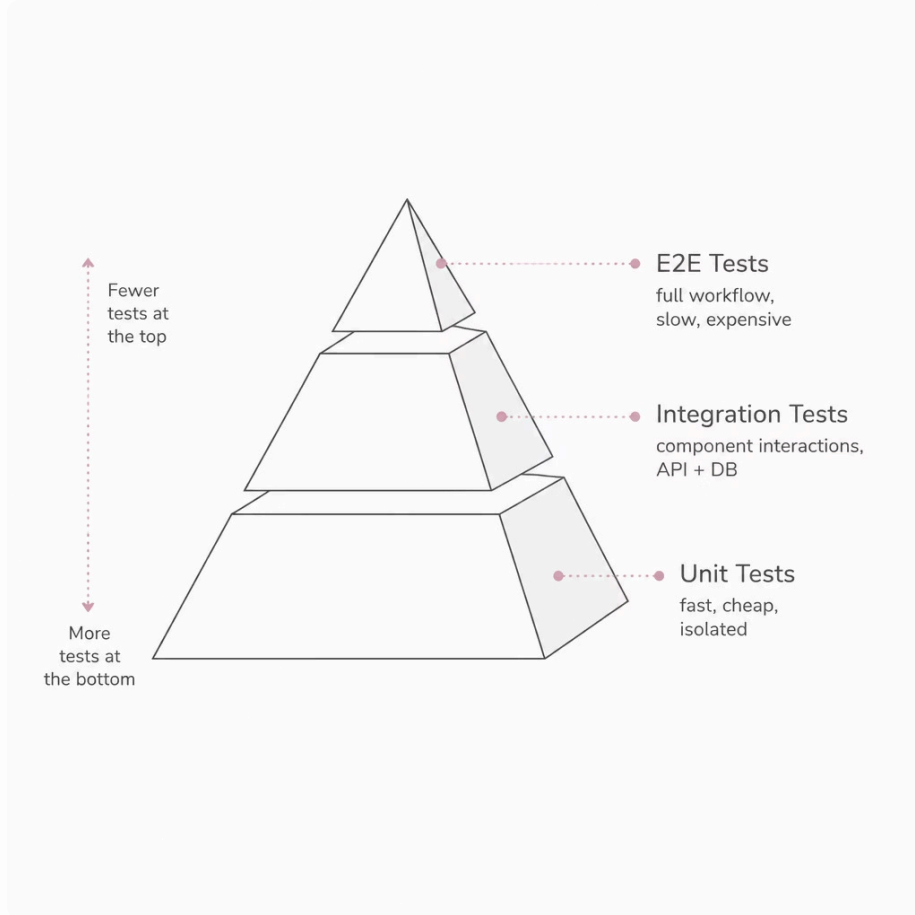
Does it still work after changes?

Catch regressions early so refactoring doesn't break existing features.



Core goal: Reduce risk before deployment — not after.

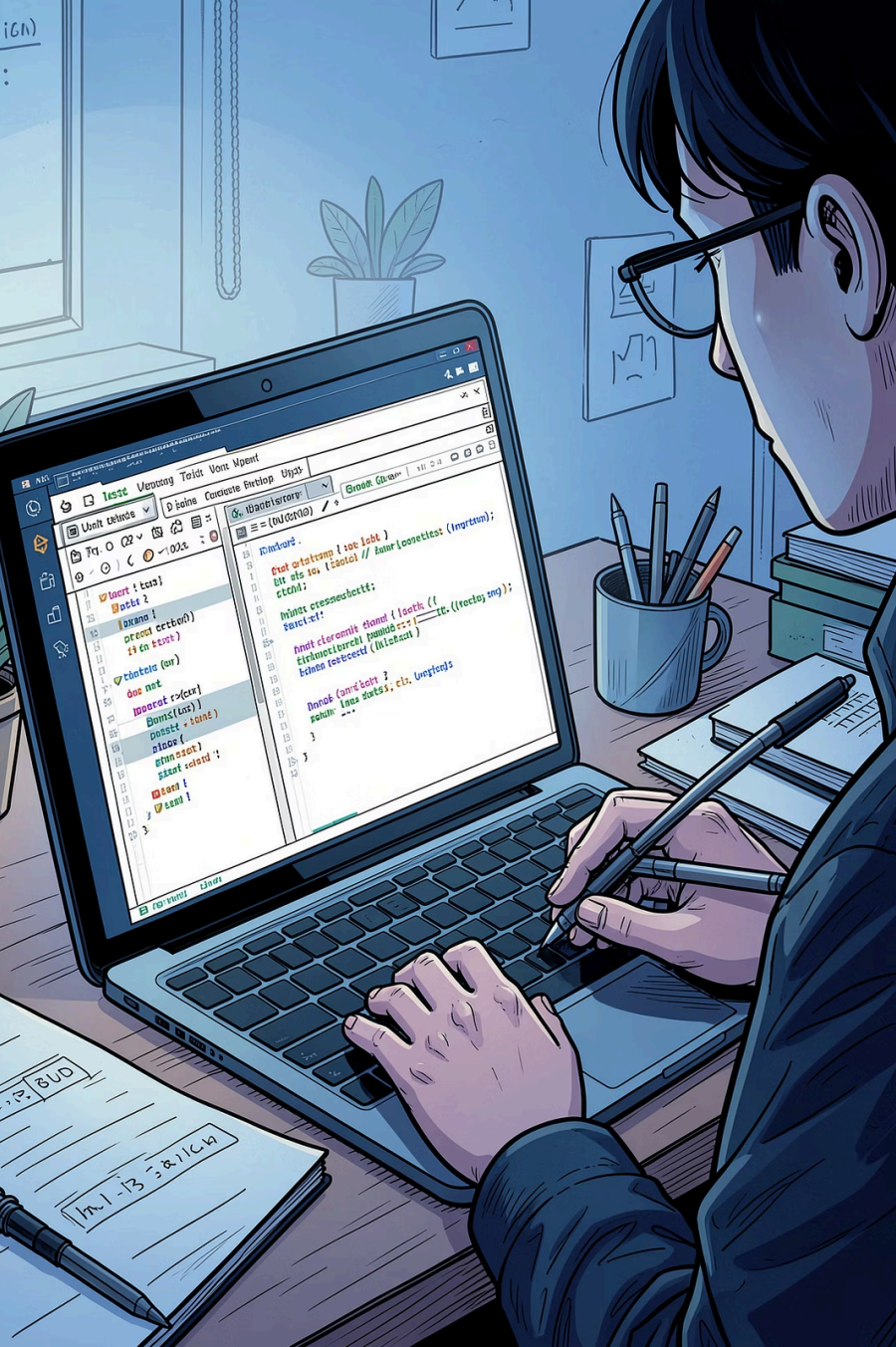
The Testing Pyramid



More at the bottom. Fewer at the top.

The pyramid reflects a practical tradeoff: **fast tests are cheap**, slow tests are expensive. Stack your test suite accordingly.

- **Unit tests** – the foundation. Fast, isolated, automated.
- **Integration tests** – verify component interactions.
- **E2E tests** – simulate real user flows. Use sparingly.



Unit Testing

Unit tests verify a **small, isolated piece of code** — a function, method, or class — without external dependencies like databases or APIs.

What to test

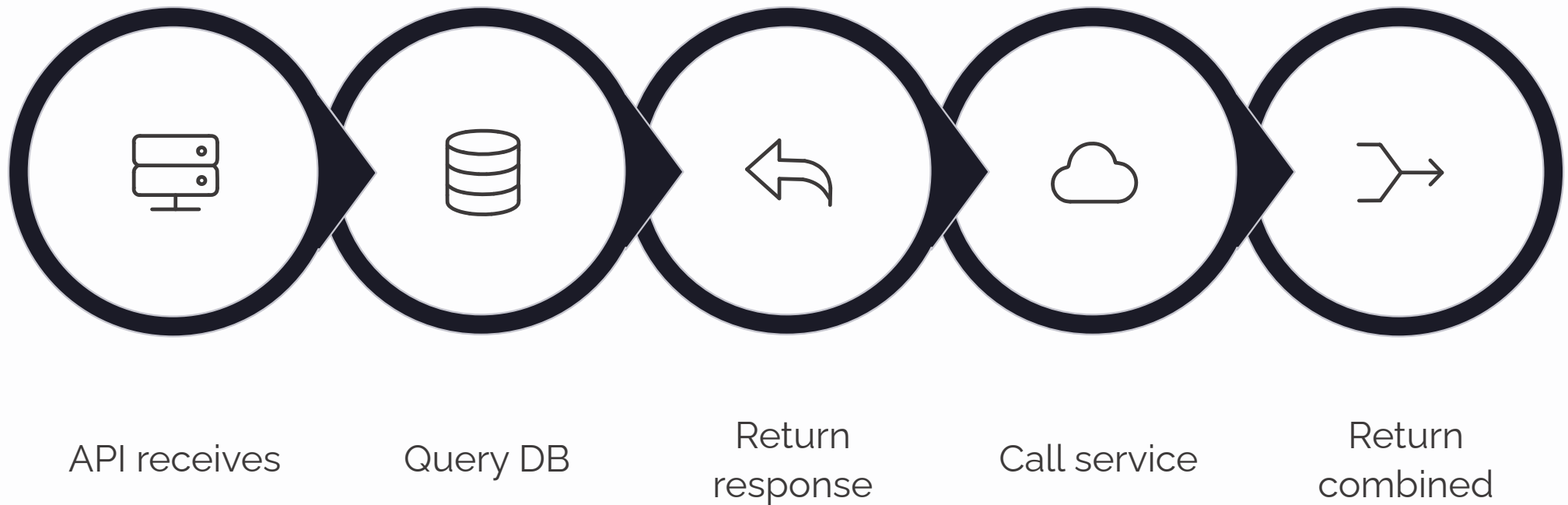
- Validate asset type
- Generate asset ID
- Enforce business rules

Why they're valuable

- ✓ Fast — milliseconds to run
- ✓ Cheap — no infrastructure needed
- ✓ Easy to automate in CI/CD

Integration Testing

Integration tests verify that **components work together correctly** — APIs talking to databases, services calling each other. They catch failures that unit tests can't see.



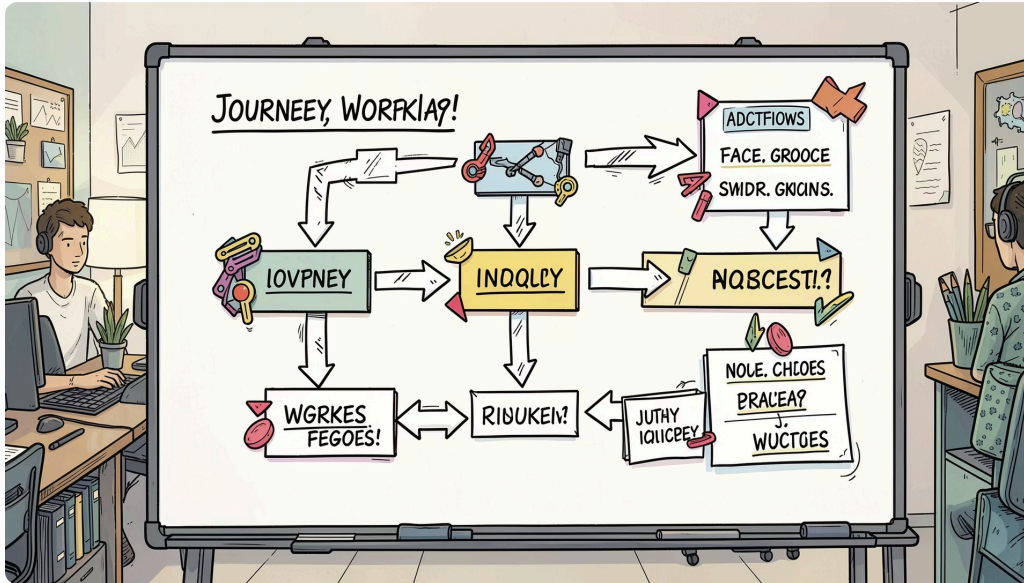
Key question: **Do my components communicate correctly?** Integration tests surface issues with connection strings, serialization, authentication, and data contracts between services.

TOP OF THE PYRAMID

End-to-End (E2E) Testing

E2E tests simulate a **complete real-world workflow** from start to finish, exercising the entire stack as a user would experience it.

⚠ Most realistic — but also the slowest and most expensive to maintain. Use strategically.



01

Create Asset

Submit a valid request through the API

02

Store in Database

Verify persistence layer works correctly

03

List Assets

Confirm data is retrievable and accurate

What Should We Test?

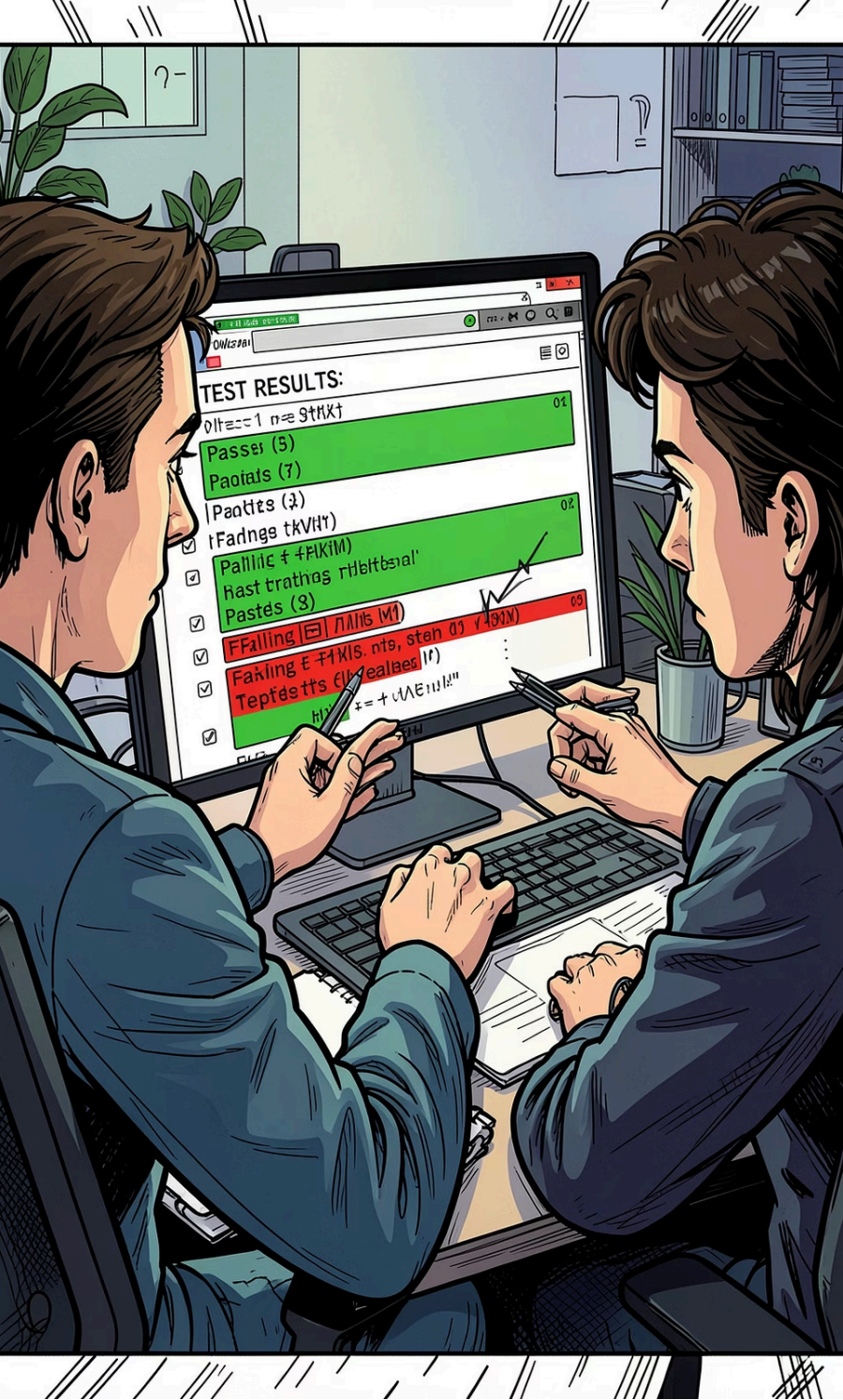
✓ Focus Here

- Business logic — the rules that drive your app
- Input validation — reject bad data early
- Edge cases — empty strings, nulls, boundaries
- Error handling — what happens when things fail

✗ Skip These

- Simple getters and setters
- Language features (e.g., if statements)
- Third-party library internals
- Generated code or framework plumbing

Test **your** logic, not someone else's.



Writing Good Test Cases

Good tests cover **failure scenarios**, not just the happy path. Think like an attacker — or an unlucky user.

- 1** Valid Request
Correct fields, valid types, expected output returned
- 2** Empty Name
Required field missing — should reject with clear error
- 3** Invalid Asset Type
Unsupported enum value — should not be accepted
- 4** Duplicate Asset
Same identifier twice — should handle conflict gracefully

Testing in Mini ASM

Apply the pyramid to your project. Start with unit tests, add integration coverage, then layer in E2E for critical paths.

Unit Tests

- Asset validation logic
- Asset service business rules
- ID generation functions

Integration Tests

- API ↔ Database queries
- API ↔ Service calls
- Authentication flows

E2E Tests

- Create Asset → Verify
- List Assets → Check Output
- Full user workflow

✅ **Start small. Automate over time.** A growing test suite compounds in value with every release.

Key Takeaways

- Testing reduces risk
Catch bugs before they reach production — not after.
- Unit tests are your foundation
They should make up the majority of your test suite.
- Test business logic first
That's where your actual risk lives.
- Validate failure scenarios
Happy-path tests alone won't protect you.
- Automate everything
Manual testing doesn't scale. CI/CD pipelines do.

The rule: If it's important, it should be tested.